



编译原理¹

Compiler Principles

Lecture 8

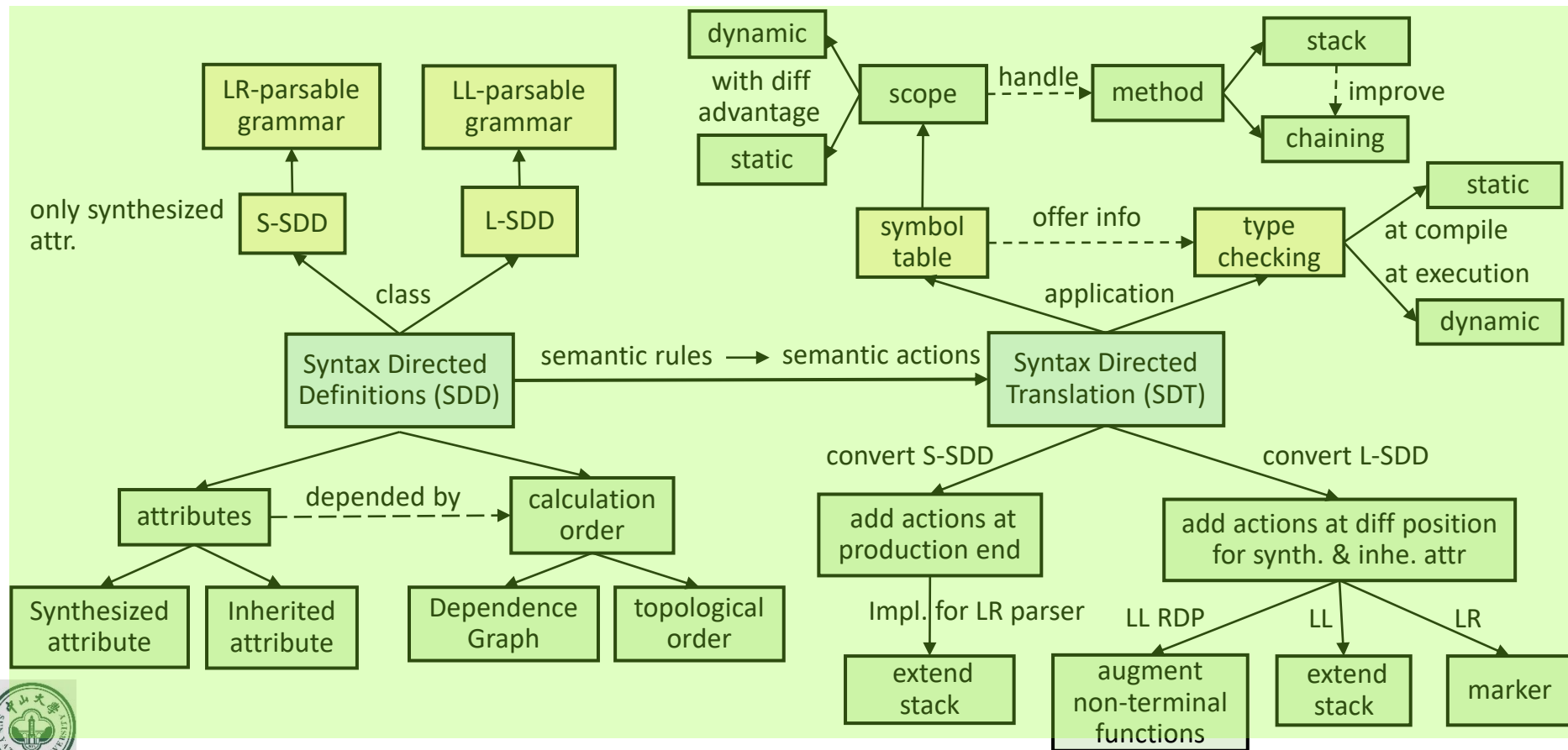
Intermediate Code: Intro & IR

赵帅²

计算机学院³

中山大学⁴

Revisit¹



Compilation Phases[编译阶段]¹



Character Stream
(Source Code)

Lexical Analyzer

token | stream

Syntax Analyzer

syntax | tree

Semantic Analyzer

syntax | tree

Front End

**Intermediate
Code Generation**

intermediate | representation

Code Optimizer

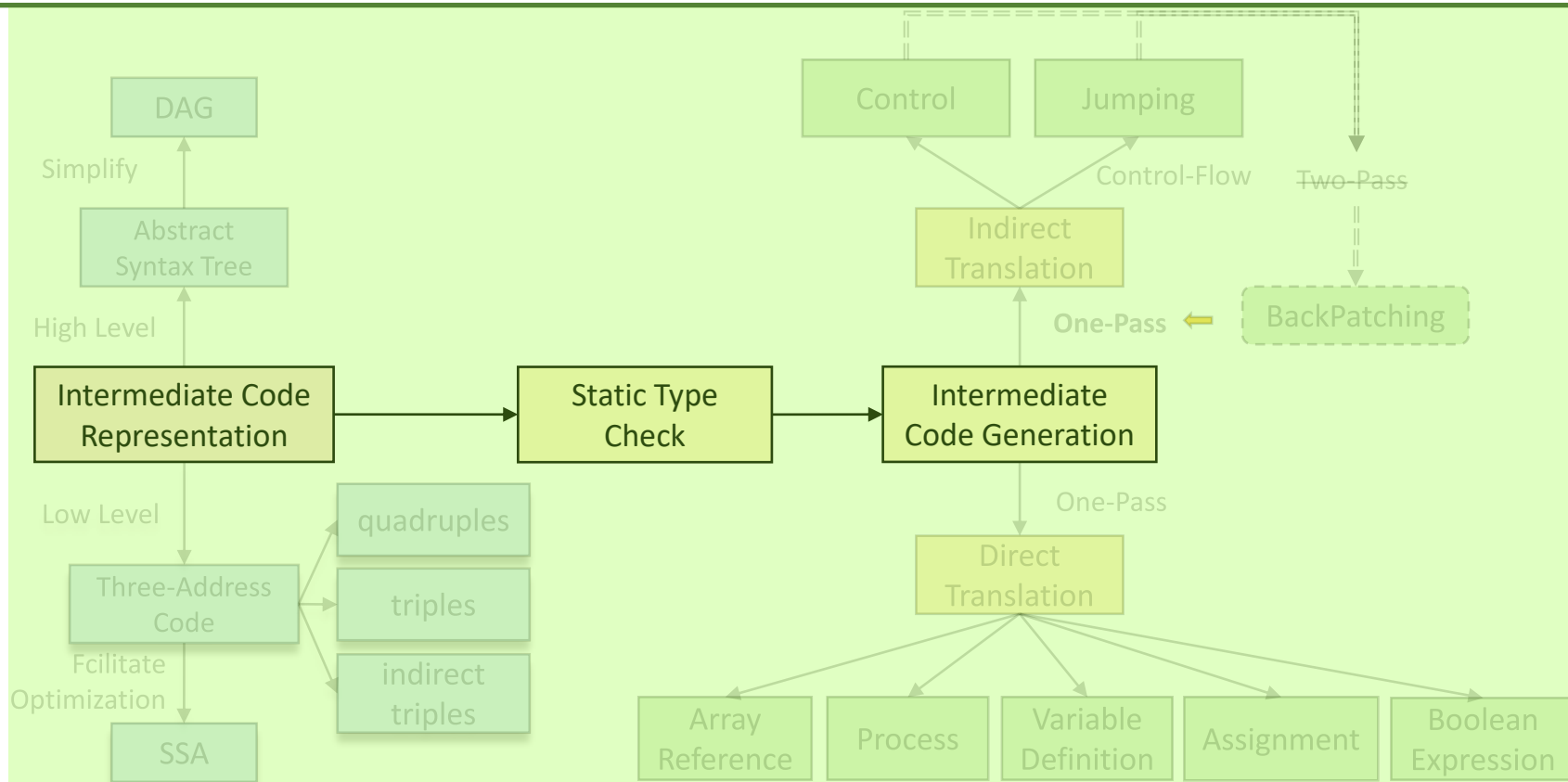
intermediate | representation

Code Generator

Back End

target-machine code
(executable)

Intermediate Code[中间代码生成]¹



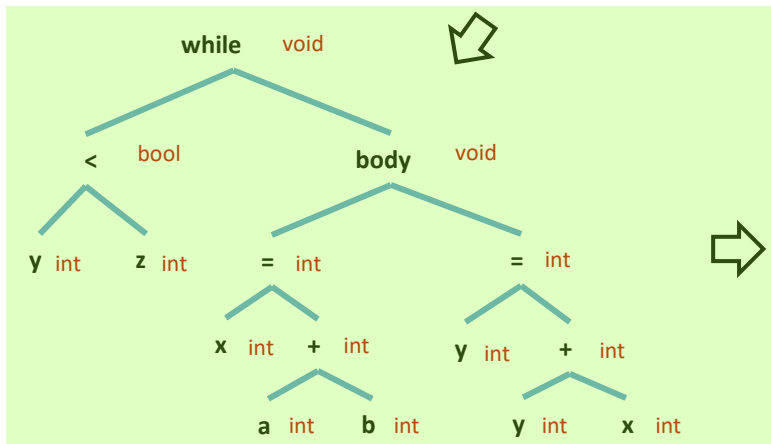
Compilation Phases[编译阶段]



```
while (y<z){  
  int x = a + b;  
  y += x;  
}
```

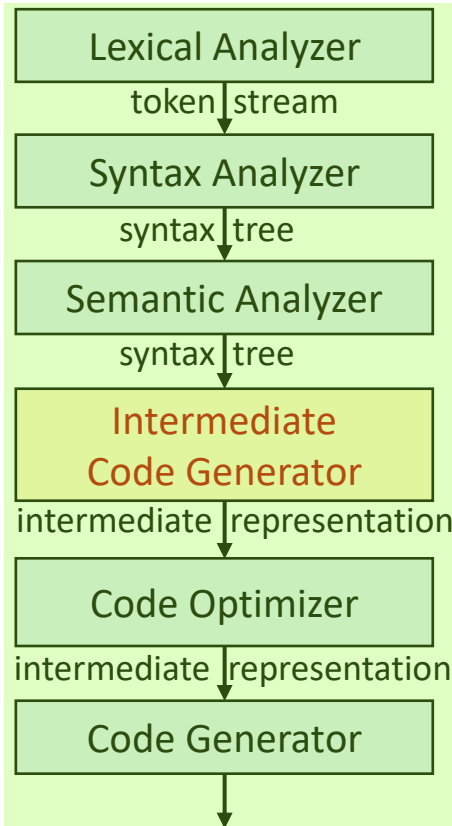


(keyword, while)	(id, b)
(id, y)	(sym, ;)
(sym, <)	(id, y)
(id, z)	(sym, +=)
(id, x)	(id, x)
(id, a)	(sym, ;)
(sym, +)	



Annotated AST/Decorated AST 3
[带标注的抽象语法树]

```
goto L1  
L2:  
  t1 := a + b  
  x := t1  
  t2 := y + x  
  y := t2  
L1:  
  if y < z goto L2
```



Target Machine Code

Multiple IR Levels [不同层级的中间表示]¹



- IR provides advantages [中间表示的优势]²

- ◆ Increased abstraction and cleaner separation³

- A compiler may construct a sequence of intermediate representations.⁴



- Modern compilers use different IRs at different stages.⁶

- **High-level IR** are close to the source code [接近源语言]⁷

- ◆ Example: Parse Tree, Abstract Syntax Tree [抽象语法树]⁸

- ◆ Language **dependent** (a high-level IR for each language)⁹

- ◆ Purpose: semantic analysis of program¹⁰

Multiple Level IR[不同层级的中间表示]¹



- **Low-level IR** are close to assembly [接近汇编]²

- ◆ E.g., three address code (TAC) [三地址码], static single assignment [静态单赋值]³
- ◆ Essentially an instruction set [指令集] for an abstract machine⁴
- ◆ Language and Machine **independent** (one common IR)⁵
- ◆ Purpose: **compiler optimizations** to make code efficient⁶
 - All optimizations written in this IR is automatically **applicable to all languages and machines**⁷

- **Machine-Level IR** [机器层级]⁸

- ◆ Example: x86 IR, ARM IR, ...⁹
- ◆ Actual instructions for a concrete machine ISA [指令集架构]¹⁰
- ◆ **Machine dependent** (a machine-level IR for each ISA)¹¹
- ◆ Purpose: code generation, CPU register allocation, etc¹²

Multiple Level IR¹[不同层级的中间表示]



- Possible to have only one IR (AST) — some compilers follows this²

- ◆ Generate **machine code from AST** after semantic analysis [AST直接到机器码]³

- ◆ Makes sense *if compilation time is the primary concern* (e.g., JIT)⁴

- Skip the IR generation step

• Why multiple IRs?⁵

1. Better to have an appropriate IR **for the task at hand** [针对性]⁶

- Semantic analysis much easier with high-level IR (AST)
- Compiler optimizations much easier with low-level IR (TAC)
- Register allocation only possible with machine-level IR (ISA)

Multiple Level IR[不同层级的中间表示]¹



- Why multiple IRs?²

2. Easier to add a new front-end (language) or back-end (ISA) [易于扩展]³

- Front-end: a new AST → low-level IR converter⁴

- Back-end: a new low-level IR → machine IR converter⁵

- Low-level IR acts as a bridge between multiple front-ends and back-ends, such that they can be reused⁶

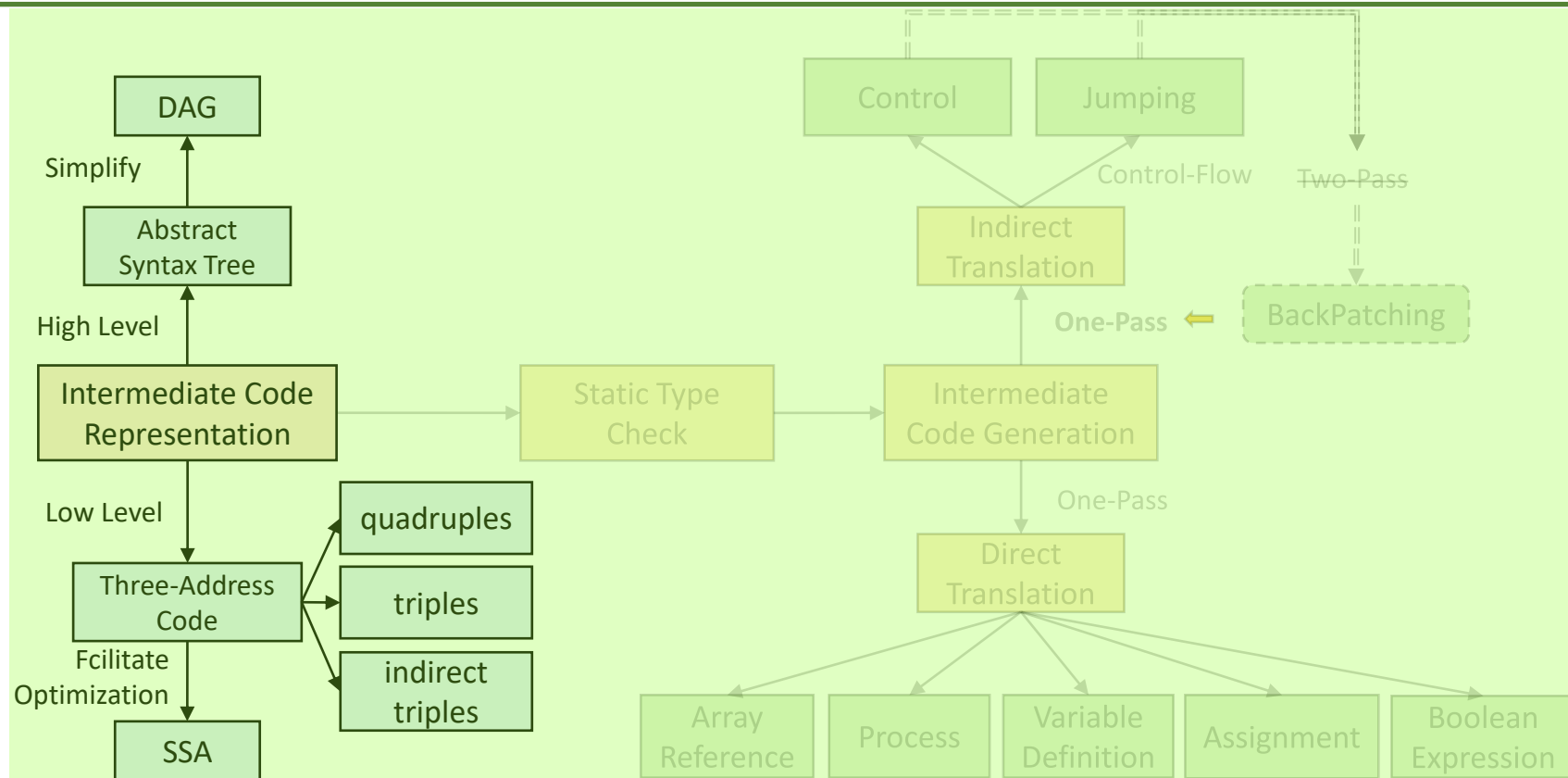
- If one IR (AST), and adding a new front-end...⁷

- ◆ Reimplement all compiler optimizations for new AST⁸

- ◆ A new AST → machine code converter for each ISA⁹

- ◆ Same goes for adding a new back-end¹⁰

Intermediate Code[中间代码生成]¹



2



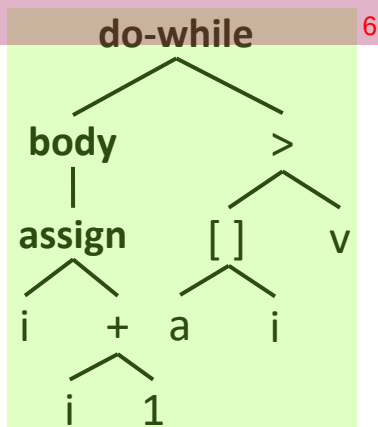
Intermediate Representation[中间表示]¹

• Two Most important IR:²

◆ **Trees** [树形结构], including parse trees and (abstract) syntax trees [语法分析树和抽象语法树]³

◆ **Directed Acyclic Graph (DAG)** [有向无环图]⁴

◆ **Linear representations** [线性表示形式], especially “**three-address code**” [三地址代码]⁵



1: $i = i + 1$ ⁷

2: $t_1 = a[i]$ ⁸

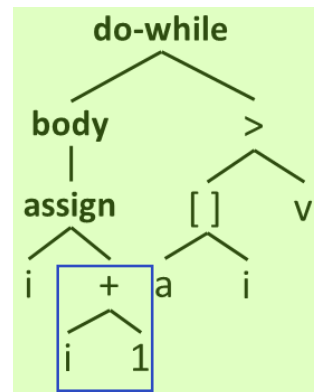
3: if $t_1 < v$ goto 1⁹

Fig. Two forms of intermediate code for “do $i=i+1$; while($a[i]<v$);”¹⁰

Three-Address Code¹[三地址代码]



- At most one operator on the right side of an instruction in three-address code, e.g., $x + y * z$ translated into $t_1 = y * z$ $t_2 = x + t_1$ ²
- Generic form is $X = Y \text{ op } Z$ [最多3个操作数]³
 - ◆ where X, Y, Z can be variables, constants, or compiler-generated temporaries holding intermediate values.⁴
- Characteristics [特性]⁵
 - ◆ a linearized representation of a syntax tree or a DAG.⁶
 - ◆ Assembly code for an “abstract machine”⁷
 - ◆ Long expressions are converted to multiple instructions⁸
 - ◆ Control flow statements are converted to jumps [控制流->跳转]⁹
 - ◆ Machine independent¹⁰



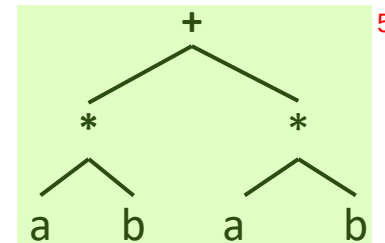
- Design goal: for easier machine-independent optimization¹²

Three-Address Code[三地址代码]¹

• Example: $a * b + a * b$ is translated to $t_1 = a * b$ ²

$t_2 = a * b$ ³

$t_3 = t_1 + t_2$ ⁴



◆ t_1, t_2, t_3 are temporary variables⁶

◆ Can be generated through a depth-first traversal of AST, and internal nodes in AST are translated to temporary variables⁷

• The repetition of $a * b$ can be eliminated by a compiler optimization called common subexpression elimination (CSE)[通用子表达式消除]⁸

$t_1 = a * b$ ⁹
 $t_3 = t_1 + t_1$

• Using 3-address code rather than AST makes it:¹⁰

• Easier to spot optimization opportunities¹¹

• Easier to manipulate IR.¹²

Addresses in three-address code[地址]¹

- An address can be one of the following:²

- ◆ **A name**[名字]. For convenience, we allow source-program names to appear as address in three-address code.³

- ◆ **A constant**[常量]. In practice, a compiler must deal with many different types of constants and variables.⁴

- ◆ **A compiler-generated temporary**[编译器生成的临时变量]. Creating a distinct name each time a temporary is needed [在每次需要临时变量时产生一个新名字是必要的], especially in optimizing compilers.⁵

- These temporaries **can be combined, if possible**, when registers are allocated to variables.⁶

Three-Address Instruction Form[三地址指令形式]¹

1. Assignment instructions [二元赋值]²

◆ $x = y \text{ op } z$, op is a **binary arithmetic**[双目算术符] or **logical operation** [逻辑运算符]³

2. Assignment instructions [一元赋值]⁴

◆ $x = \text{op } y$, where op is a **unary operation**[单目运算符]. Essential unary operations include unary minus, logical negation, shift operators, and conversion operators.⁵

3. Copy instructions [复制]⁶

◆ $x = y$, where x is assigned the value of y [把 y 的值赋给 x].⁷

4. Unconditional Jump instructions [无条件转移指令]⁸

◆ **goto L**: the three-address instruction with **label L** is the next to be executed.⁹

5. Conditional Jump instructions [条件转移指令]¹⁰

◆ **if x goto L** **if False x goto L** **if (x relop y) goto L**¹¹

◆ where **relop** is a relational operator such as $==$, $>$, $<$, etc.¹²

Three-Address Instruction Form[三地址指令形式]¹

6. Procedure calls [程序调用]²

◆ *param x* for parameters [参数传递];³

◆ *call p,n* for procedure call – *p*: the procedure, *n*: the number of params.⁴

param x₁

param x₂

...

param x_n

call p,n

param x1

param x2

param x3 call

foo, 3

◆ Part of a call of the procedure $p(x_1, x_2, \dots, x_n)$.⁵

7. Procedure calls return statement [程序调用返回]⁶

◆ *return y*, *y* representing a returned value, is optional.

Three-Address Instruction Form[三地址指令形式]¹

8. Indexed copy instructions [带下标的复制指令]²

◆ $x = y[i]$ $x[i] = y$ ³

◆ $x = y[i]$ sets x to the value in the location i memory units beyond location y . ⁴

◆ $x[i] = y$ sets the contents of the location i units beyond x to the value of y . ⁵

9. Address and pointer assignments instructions. [地址及指针赋值指令]⁶

◆ $x = \&y$ a pointer x is set to address of y [取址] ⁷

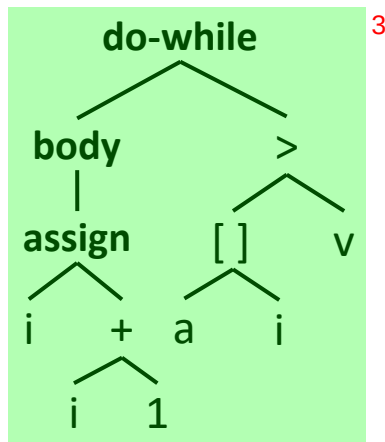
◆ $x = *y$ x is set to the value of location pointed to by pointer y [y 地址指向的值赋给 x] ⁸

◆ $*x = y$ location pointed to by x is assigned y [y 的值赋给 x 地址指向的位置] ⁹

Example¹

```
do {  
    i = i + 1;  
} while(a[i] < v)
```

Source program²



Syntax tree⁴

L: $t_1 = i + 1$

$i = t_1$

$t_2 = \&a$

$t_3 = \text{sizeof}(\text{int})$

$t_4 = t_3 * i$

$t_5 = t_2 + t_4$

$t_6 = *t_5$

if $t_6 < v$ goto L

Three-address code⁷

$a[i]$

Implementation of Three-address Code[实现]¹

- Three representations. (and more)²

- ◆ **quadruples.** [四元式]³

- ◆ **triples.** [三元式]⁴

- ◆ **indirect triples.** [间接三元式]⁵

- Trade-offs between **space**, **speed**, **ease of manipulation**.⁶

- **Quadruples.** [四元式]⁷

- ◆ A quadruple has four fields, which we call **op**, **arg₁**, **arg₂**, **result**.⁸

- ◆ Examples & some exceptions:⁹

- $x = y + z \Rightarrow (+, y, z, x)$ ¹⁰

- Note that for a copy statement like **x = y**, **op is =**, while for most other operations, the assignment operator is implied. [隐含表示的]¹¹

Quadruples¹[四元式]

• Quadruples. [四元式]²

◆ Examples & some exceptions:³

□ $x = \text{minus } y \Rightarrow (\text{minus}, y, , x)$ ⁴

□ Instructions with unary[一元] operators like $x = \text{minus } y$ or $x = y$ do not use arg_2 .⁵

□ Operators like `param` use neither arg_2 nor `result`.⁶

□ $(\text{param}, x1, ,)$ ⁷

□ $\text{goto } L \Rightarrow (\text{goto}, , , L)$ ⁸

□ Conditional and Unconditional jumps put the target label in `result`.⁹

[条件或非条件转移指令将目标标号放入result字段]

Quadruples¹[四元式]



• **Example:** $a = b * (-c) + b * (-c)$ ²

◆ The special operator `minus` is used to distinguish the unary minus operator, as in “-c”, from the binary minus operator, as in “b-c”.³

$t_1 = \text{minus } c$ ⁴

$t_2 = b * t_1$ ⁵

$t_3 = \text{minus } c$ ⁶

$t_4 = b * t_3$ ⁷

$t_5 = t_2 + t_4$ ⁸

$a = t_5$ ⁹

Three-address code¹¹

	op	arg ₁	arg ₂	result
(0)	minus	c		t ₁
(1)	*	b	t ₁	t ₂
(2)	minus	c		t ₃
(3)	*	b	t ₃	t ₄
(4)	+	t ₂	t ₄	t ₅
(5)	=	t ₅		a
Quadruples				21

Triples[三元式]¹

- A triple has only three fields, which we call **op**, **arg₁**, **arg₂**.²

- ◆ Quadruple without the result field.³

- $x = y + z \Rightarrow (+, y, z)$ ⁴

- the assignment operator (**x=**) is implied⁵

- ◆ Result field is implicitly **index** of instruction.⁶

- ◆ Result referred to **by index** of instructions computing it.⁷

- See example in the next slide⁸

Triples[三元式]¹



• **Example:** $a = b * (-c) + b * (-c)$ ²

◆ The copy statement $a=t_5$ is encoded in the triple representation by placing a in the arg_1 field and (4) in the arg_2 field.³

	op	arg ₁	arg ₂	result
(0)	minus	c		t_1
(1)	*	b	t_1	t_2
(2)	minus	c		t_3
(3)	*	b	t_3	t_4
(4)	+	t_2	t_4	t_5
(5)	=	t_5		a

Quadruples⁵

	op	arg ₁	arg ₂
(0)	minus	c	
(1)	*	b	(0)
(2)	minus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	=	a	(4)

Triples

More About Triples[三元式]¹



• How can the following statements be expressed in triple?²

◆ Array location (e.g. $x[i] = y$)³

◆ Pointer location (e.g. $*(x+i) = y$)⁴

◆ Struct field location (e.g. $x.i = y$)⁵

• Example: $x[i] = y$ ⁶

◆ Requires **two entries** in the triple structure.⁷

◆ is translated to:⁸

	op	arg ₁	arg ₂
(0)	[]	x	i
(1)	=	(0)	y

⁹

Complex LHS may require more triples to compute address¹⁰

// Compute address of $x[i]$ location¹¹

// Assign y to that location¹²

Problems About Triples¹_[三元式]



• Problem with triples²

◆ In code optimization, instructions are often moved around.³

◆ With triples, the result of an operation is referred to by its position, so **moving**⁴ an instruction may require us to change all references to that result.

⁵

	op	arg ₁	arg ₂	result
(0)	minus	c		t ₁
(1)	*	b	t ₁	t ₂
(2)	minus	c		t ₃
(3)	*	b	t ₃	t ₄
(4) (2)	+	t ₂	t₄ t ₂	t ₅
(5) (3)	=	t ₅		a

⁸

	op	arg ₁	arg ₂
(0)	minus	c	
(1)	*	b	(0)
(2)	minus	c	
(3)	*	b	(2)
(4) (2)	+	(1)	(3) (1)
(5) (3)	=	a	(4)

⁶
$$t_1 = a * b$$
⁷~~$$t_2 = a * b$$~~
⁹
$$t_3 = t_1 + t_2$$
¹⁰
$$t_1 = a * b$$
¹¹
$$t_3 = t_1 + t_1$$

CSE

Problems About Triples¹_[三元式]



- Problem with triples²

- ◆ In code optimization, instructions are often moved around.³

- ◆ With triples, the result of an operation is referred to by its position, so moving an instruction may require us to change all references to that result.⁴

	op	arg ₁	arg ₂	result ⁵
(0)	minus	c		t ₁
(1)	*	b	t ₁	t ₂
(2)	+	t ₂	t ₂	T ₅
(3)	=	t ₅		a

	op	arg ₁	arg ₂	⁶
(0)	minus	c		
(1)	*	b	(0)	
(2)	+	(1)	(1)	
(3)	=	a	(4)	✗

Instruction (3) refers to (4) which is no longer there.⁷

Three-Address Code[三地址代码] (Recap)¹



- Generic form is $X = Y \text{ op } Z$ [最多3个操作数]²
- Three representations. (and more)³

◆ quadruples [四元式]

□ $x = y + z \Rightarrow (+, y, z, x)$

◆ triples [三元式]

□ $x = y + z \Rightarrow (1) (+, y, z)$, use index for the result

◆ indirect triples [间接三元式]

□ use an **index list** for TAC execution

step	instruction	a triple 'database'			
		index	op	arg ₁	arg ₂
0	(0)				
1	(1)	(0)	+	a	b
2	(2)	(1)	*	(0)	c
3	(0)	(2)	=	x	(1)
4	(3)	(3)	/	d	(0)
5	(4)	(4)	=	y	(3)

• Single Static Assignment⁵

- Every variable is assigned exactly once statically[仅一次]⁶

Indirect Triples¹[间接三元式]



- *The problem does not occur with **indirect triples**.*²

- Indirect triples consist of a listing of pointers to triples, rather than a³
listing of triples themselves.⁴

*Triples are stored in a triple 'database'*⁶

step	instruction
0	(0)
1	(1)
2	(2)
3	(3)
4	(4)
5	(5)

index	op	arg ₁	arg ₂
(0)	minus	c	
(1)	*	b	(0)
(2)	minus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	=	a	(4)

Indirect Triples¹[间接三元式]



- After CSE, empty entries in database can be reused²

◆ Code in triple database becomes **non-contiguous** over time³

◆ That's fine since the listing is the code, not the database⁴

*Triples are stored in a triple 'database'*⁵

<i>step</i>	<i>instruction</i>
0	(0)
1	(1)
2	(4)
3	(5)

index	op	arg ₁	arg ₂
(0)	minus	c	
(1)	*	b	(0)
(2)		empty	
(3)		empty	
(4)	+	(1)	(1)
(5)	=	a	(4)

Indirect Triples¹[间接三元式]



- Another Example: $x = (a+b)*c$; $y = d/(a+b)$ ²

step	instruction
0	(0)
1	(1)
2	(2)
3	(0)
4	(3)
5	(4)

index	op	arg ₁	arg ₂
(0)	+	a	b
(1)	*	(0)	c
(2)	=	x	(1)
(3)	/	d	(0)
(4)	=	y	(3)

- With indirect triples, an optimizing compiler can move an instruction by reordering the instruction list, without affecting the triples themselves.⁵

Single Static Assignment¹[静态单赋值]

- Every variable is assigned exactly once statically[仅一次]²

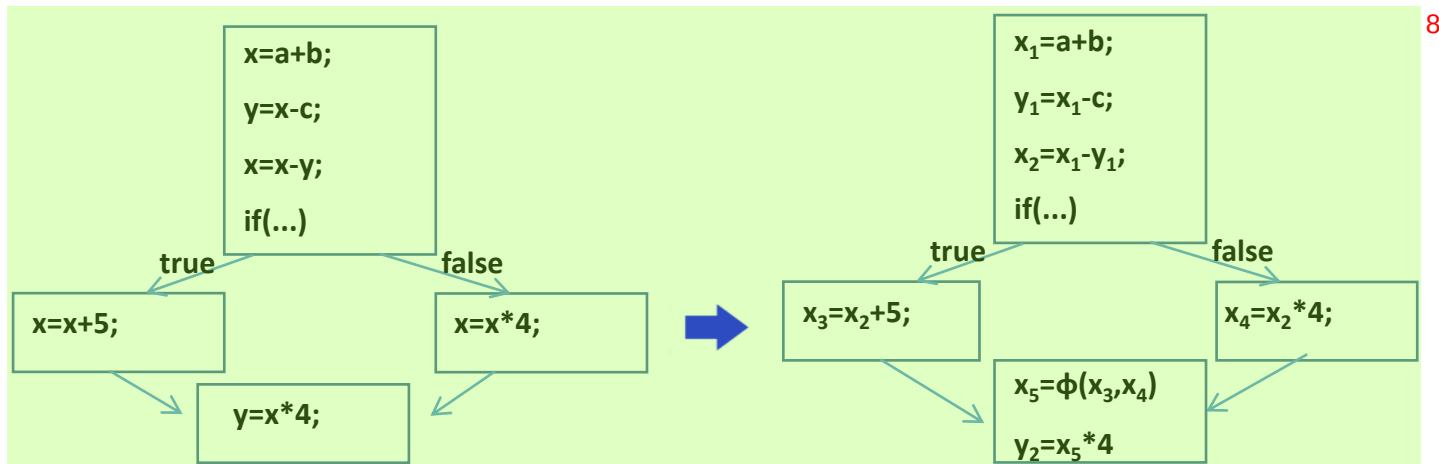
◆ Give variable a different version name on every assignment³

□ e.g. $x \rightarrow x_1, x_2, \dots, x_5$ for each static assignment of x ⁴

◆ Now value of each variable guaranteed not to change⁵

◆ On a control flow merge, ϕ -function combines two versions⁶

□ e.g. $x_5 = \phi(x_3, x_4)$: means x_5 is either x_3 or x_4 ⁷



Benefits of SSA¹

- **SSA** is an IR that facilitates code optimization²

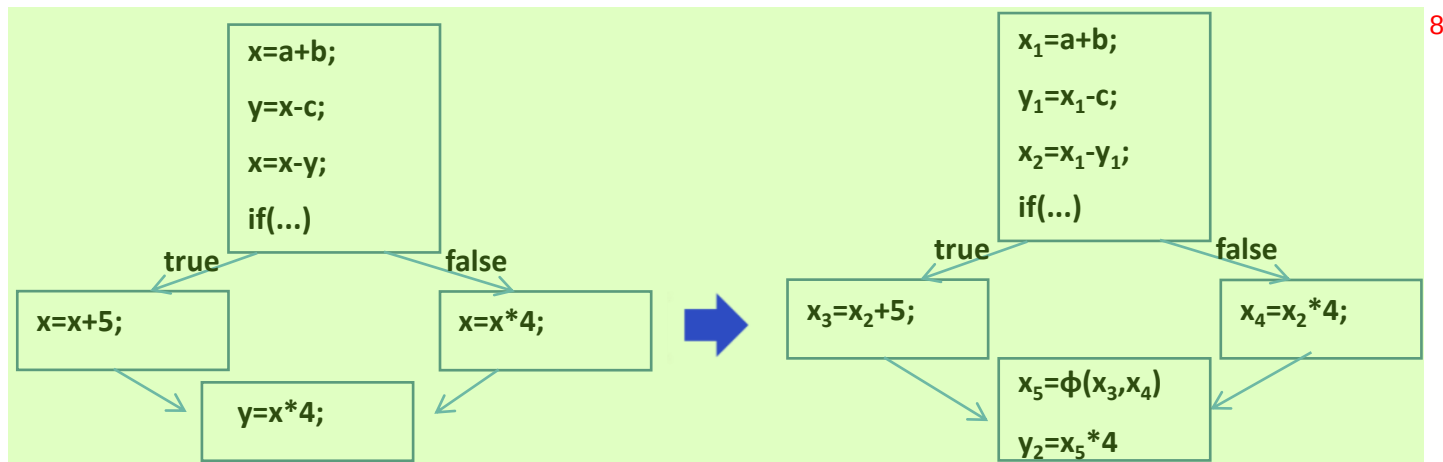
◆ SSA tells you when an optimization **should not** happen³

◆ Suppose compiler performs CSE on previous example:⁴

- Without SSA, (incorrectly) tempted to eliminate second $x * 4$ ⁵

$x = x * 4;$ $y = x * 4;$ $\rightarrow$ $x = x * 4;$ $y = x;$ ⁶

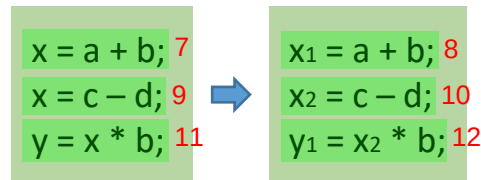
- With SSA, $x_2 * 4$ and $x_5 * 4$ are clearly different values⁷



Benefits of SSA (cont.)¹

- SSA is an IR that facilitates code optimizations²

- SSA tells you when an optimization **should** happen³
- Suppose compiler performs dead code elimination (DCE): (DCE removes code that computes dead values)⁴
- Without SSA, not very clear whether there are dead values⁵
- With SSA, **x1 is never used** and clearly a dead value⁶



- Why does SSA work so well with compiler optimizations?¹³

- SSA makes flow of values explicit in the IR¹⁴
- Without SSA, need a separate dataflow graph¹⁵
- Will discuss more in **Compiler Optimization** section¹⁶

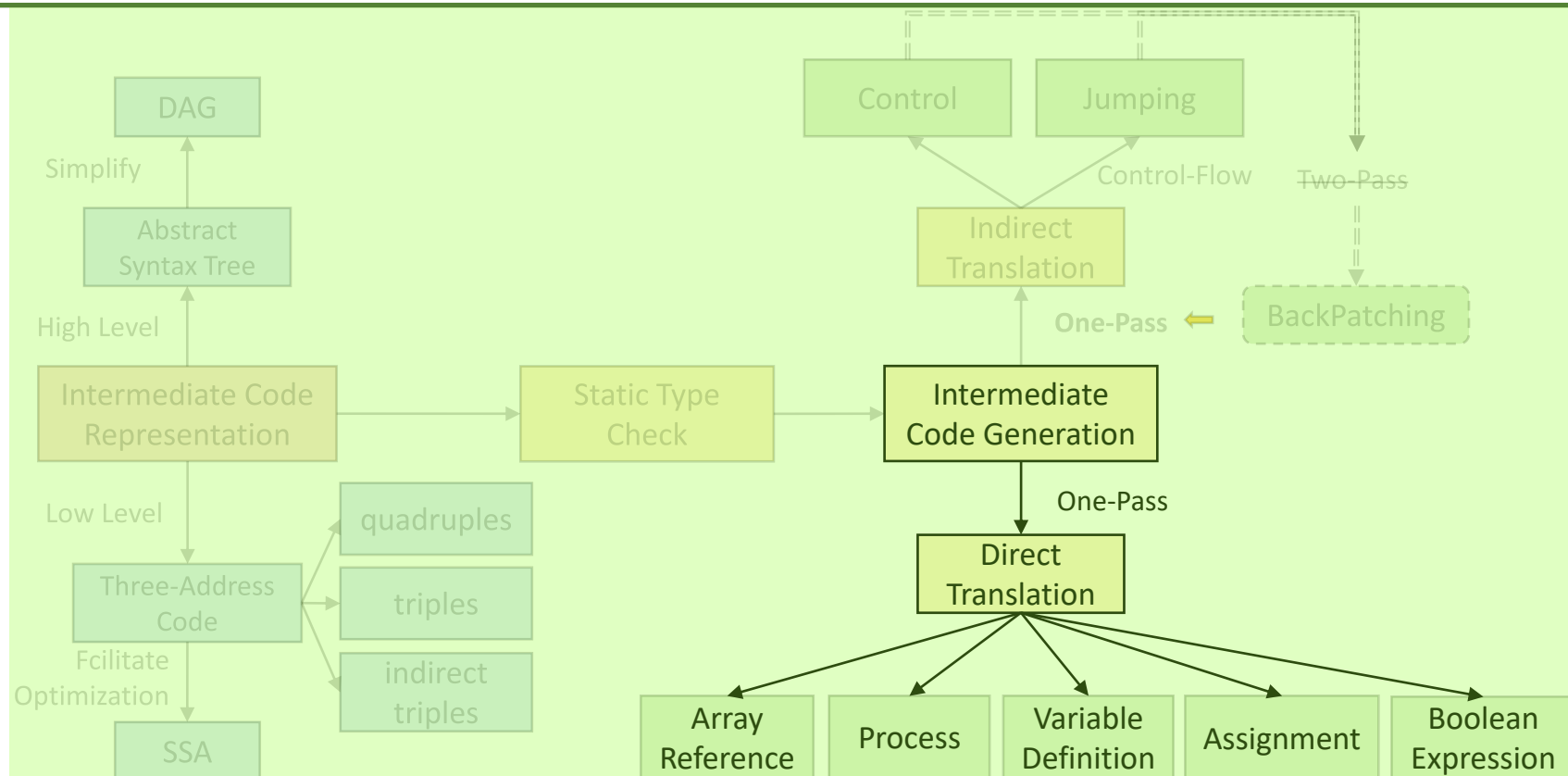
Syntax Directed Translation[语法制导翻译]



- **Syntax directed translation** can be used again for **code generation** [代码生成]²
 - ◆ Code generation is **dependent on syntax/AST**³
 - ◆ Code generation is to translate the **syntactic structures**⁴
- What language structures do we need to translate?[翻译]⁵
 - ◆ **Definitions** (variables, functions, ...) ⁶
 - ◆ **Assignment** statements ⁷
 - ◆ **Array** references ⁸
 - ◆ **Boolean** expressions ⁹
 - ◆ **Control flow** statements (if-then-else, for, etc)... ¹⁰
- We are going to use the following strategy: ¹¹
 - ◆ Specify **SDD semantic rules** (without ordering) ¹²
 - ◆ Convert SDD rules to **SDT actions** (with ordering) ¹³



Intermediate Code[中间代码生成]¹



Code Generation Overview[代码生成]¹

- Program code is a collection of functions²
 - ◆ By now, all functions are listed in symbol table³
- Goal is to generate code for each function in that list⁴
- Generating code for a function involves two steps:⁵
 - ◆ Processing variable definitions[变量定义] -> Laying out variables in memory⁶
 - ◆ Processing statements[语句] -> Generating instructions for statements⁷
 - Assignments, array references, boolean expressions, control-flow statements⁸
- We will start with variable definitions⁹

Processing Variable Definitions[变量定义]



- To lay out a variable, both **location** and **width** are needed²

- ◆ **Location**: where variable is located in memory³

- ◆ **Width**: how much space variable takes up in memory⁴

- Attributes for variable definition:⁵

- ◆ **T V**, e.g., **int x**;⁶

- ◆ **T**: non-terminal for **type** name⁷

- **T.type**: type (int, float, ...)⁸

- **T.width**: width of type in bytes (e.g., 4 for int)⁹

- ◆ **V**: non-terminal for **variable** name¹⁰

- **V.type**: type (int, float, ...)¹¹

- **V.width**: width of variable according to type¹²

- **V.offset**: offset of variable in memory¹³

- ◆ But **offset from what...**?¹⁴



Variable Location from Offset¹

- **Naive method:** reserve a **big memory** section for all data²
 - ◆ Size data section to be large enough to contain all variables³
 - ◆ Location = **var offset** + **base of data section**⁴
- Naive method **wastes a lot of memory**⁵
 - ◆ Vars with limited scope **only live briefly in memory**⁶
 - E.g., function variables last only for duration of call⁷
- **Solution:** allocate memory for each scope[域内]⁸
 - ◆ **Allocate** when **entering** scope, **free** when **exiting** scope⁹
 - ◆ Variables in the same scope are allocated / freed together¹⁰
 - ◆ Location = **var offset** + **base of scope memory** section¹¹
 - ◆ Will discuss more later in **Runtime Management**¹²



Storage Layout of Variables in a Function



- When there are **multiple variables** defined in a function²

- ◆ Compiler **lays out variables** in memory **sequentially**³

- ◆ Current offset used to place variable **x** in memory⁴

- $\text{address}(x) \leftarrow \text{offset}$ ⁵

- $\text{offset} += \text{sizeof}(x.\text{type})$ ⁶

```
void foo() {  
    int a;  
    int b;  
    long long c;  
    int d;  
}
```

Address ⁸		
0x0000	a	Offset = 0 Addr(a) $\leftarrow$ 0
0x0004	b	Offset = 4 Addr(b) $\leftarrow$ 4
0x0008	c	Offset = 8 Addr(c) $\leftarrow$ 8
0x000c	c	
0x0010	d	Offset = 16 Addr(d) $\leftarrow$ 16
		Offset = 20

More about Storage Layout¹

• Allocation alignment[对齐]²

◆ Enforce $\text{addr}(x) \% \text{sizeof}(x.\text{type}) == 0$ ³

◆ Most machine architectures are designed such that computation is most efficient at $\text{sizeof}(x.\text{type})$ boundaries⁴

□ E.g. most machines are designed to load integer values at integer word boundaries⁵

□ If not on word boundary, need to load two words with shift & concatenate → inefficient

```
void foo() {6
```

```
    char a;8      // addr(a) = 09
```

```
    int b;10     // addr(b) = 111
```

```
    int c;12     // addr(c) = 515
```

```
    long long d;17 // addr(d) = 9
```

```
}
```



```
void foo() {7
```

```
    char a;14     // addr(a) = 013
```

```
    int b;18     // addr(b) = 419
```

```
    int c;20     // addr(c) = 8
```

```
    long long d;21 // addr(d) = 16
```

```
}
```

```
#include<stdio.h>22
```

```
struct{  
    char x; int y;  
}Test;
```

```
int main() {  
    printf("%d\n",sizeof(Test));  
    return 0;  
}
```

What is the output?
Not aligned: 5, Aligned: 8

²³

40

Code Generation¹[代码生成]

- We will use the **syntax-directed formalisms** to specify translation²

- ◆ Variable definitions[变量定义] -> Recall semantic analysis & symbol table³

- ◆ Assignment[赋值]⁴

- ◆ Array references[数组引用]⁵

- ◆ Boolean expressions[布尔表达式]⁶

- ◆ Control-flow statements[控制流语句]⁷

- To generate three-address codes (TACs)⁸

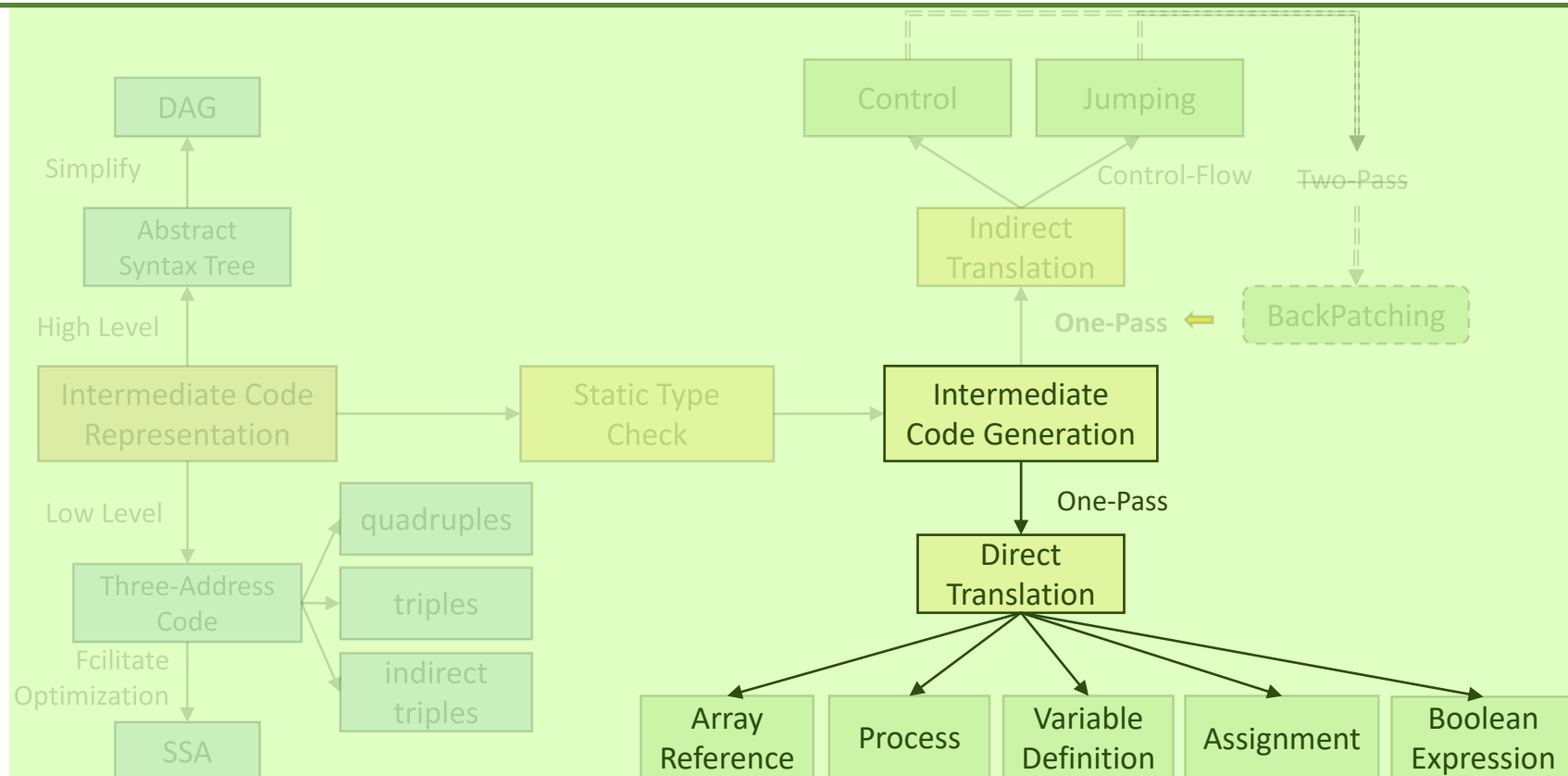
- ◆ Lay out variables in memory⁹

- ◆ Generate TAC for any subexpressions or substatements¹⁰

- ◆ Using the result, generate TAC for the overall expression¹¹



Intermediate Code[中间代码生成]¹



Code Generation¹[代码生成]

- SSA (Single Static Assignment)²

- We will use the **syntax-directed formalisms** to specify translation³

 - ◆ Variable definitions[变量定义] -> Recall semantic analysis & symbol table⁴

 - ◆ Assignment[赋值]⁵

 - ◆ Array references[数组引用]⁶

 - ◆ Boolean expressions[布尔表达式]⁷

 - ◆ Control-flow statements[控制流语句]⁸

- To generate three-address codes (TACs)⁹

 - ◆ Lay out variables in memory¹⁰

 - ◆ Generate TAC for any subexpressions or substatements¹¹

 - ◆ Using the result, generate TAC for the overall expression¹²



CodeGen: Assignment Statement¹

- Translate into three-address code[赋值语句]²

- ◆ An expression with **more than one operator** will be translated into instructions with **at most one operator per instruction**³

- Helper functions in translation⁴

- ◆ **lookup(id)**: search **id** in symbol table, return **null** if none⁵

- ◆ **emit()/gen()**: generate three-address IR⁶

- ◆ **newtemp()**: get a new temporary location⁷

① $S \rightarrow id = E;$ ⁹

② $E \rightarrow E_1 + E_2;$ ¹¹

③ $E \rightarrow - E_1$ ¹³

④ $E \rightarrow (E_1)$ ¹⁵

⑤ $E \rightarrow id$ ¹⁷

Assignment statement:⁸

$a = b + (-c)$ ¹⁰

Three-address code:¹²

$t_1 = \text{minus } c$ ¹⁴

$t_2 = b + t_1$ ¹⁶

$a = t_2$ ¹⁸



SDT Translation of Assignment¹

- Attributes code and addr²

- ◆ S.code and E.code denote the TAC for S and E, respectively³

- ◆ E.addr denotes the address that will hold the value of E (can be a name, constant, or a compiler-generated temporary)⁴

① S → id = E; ⁶

② E → E₁ + E₂; ⁸

③ E → - E₁ ¹⁰

④ E → (E₁) ¹²

⑤ E → id ¹⁴

Assignment statement: ⁵

a = b + (-c) ⁷

Three-address code: ⁹

t₁ = minus c ¹¹

t₂ = b + t₁ ¹³

a = t₂ ¹⁵

① S → id = E; { p = lookup(id.lexeme); if !p then error; S.code = E.code || gen(p '=' E.addr); }

② E → E₁ + E₂; { E.addr = newtemp(); E.code = E₁.code || E₂.code || gen(E.addr '=' E₁.addr '+' E₂.addr); }

③ E → - E₁ { E.addr = newtemp(); E.code = E₁.code || gen(E.addr '=' 'minus' E₁.addr); }

④ E → (E₁) { E.addr = E₁.addr; E.code = E₁.code; }

⑤ E → id { E.addr = lookup(id.lexeme); if !E.addr then error; E.code = ";" }

¹⁶

Incremental Translation[增量翻译]¹

• Generate only the new three-address instructions²

- ◆ `gen()` **not only constructs** a three-address inst, it **appends the inst** to the sequence³ of insts generated so far

- ① $S \rightarrow id = E;$
- ② $E \rightarrow E_1 + E_2;$
- ③ $E \rightarrow - E_1$
- ④ $E \rightarrow (E_1)$
- ⑤ $E \rightarrow id$

$S \rightarrow id = E; \{ p = \text{lookup}(id.\text{lexeme}); \text{if } !p \text{ then error; } S.\text{code} = E.\text{code} \parallel \text{gen}(p \neq E.\text{addr}); \}$

② $E \rightarrow E_1 + E_2; \{ E.\text{addr} = \text{newtemp}(); E.\text{code} = E_1.\text{code} \parallel E_2.\text{code} \parallel \text{gen}(E.\text{addr} \neq E_1.\text{addr} '+' E_2.\text{addr}); \}$

③ $E \rightarrow - E_1 \{ E.\text{addr} = \text{newtemp}(); E.\text{code} = E_1.\text{code} \parallel \text{gen}(E.\text{addr} \neq \text{'minus'} E_1.\text{addr}); \}$

④ $E \rightarrow (E_1) \{ E.\text{addr} = E_1.\text{addr}; E.\text{code} = E_1.\text{code}; \}$

⑤ $E \rightarrow id \{ E.\text{addr} = \text{lookup}(id.\text{lexeme}); \text{if } !E.\text{addr} \text{ then error; } E.\text{code} = ''; \}$

Code Generation¹[代码生成]

- We will use the **syntax-directed formalisms** to specify translation²

- ◆ Variable definitions[变量定义] -> Recall semantic analysis & symbol table³

- ◆ Assignment[赋值]⁴

- ◆ Array references[数组引用]⁵

- ◆ Boolean expressions[布尔表达式]⁶

- ◆ Control-flow statements[控制流语句]⁷

- To generate three-address codes (TACs)⁸

- ◆ Lay out variables in memory⁹

- ◆ Generate TAC for any subexpressions or substatements¹⁰

- ◆ Using the result, generate TAC for the overall expression¹¹



CodeGen: Array Reference¹[数组引用]

- Primary problem in generating code for array references is to determine the address of element²

- 1D array:³

```
int A[N];4
```

```
A[i] ++;5
```



◆ **base**: address of the first element⁶

◆ **width**: width of each element⁸

□ $i * \text{width}$ is the offset⁹

- Addressing an array element¹¹

◆ $\text{addr}(A[i]) = \text{base} + i \times \text{width}$ ¹²



N-dimensional Array¹

- Laying out 2D array in 1D memory²

```
int A[N1][N2]; /* int A[0..N1][0..N2] */3  
A[i1][i2] ++;
```

- Organization by row-major or column-major⁴

- ◆ C language uses row major (i.e., row by row)⁵

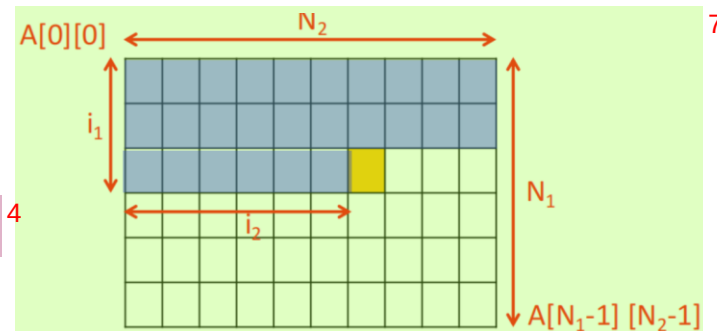
◆ $\text{addr}(A[i_1, i_2]) = \text{base} + (i_1 \times \underbrace{N_2}_{W_1} \times \text{width} + i_2 \times \underbrace{\text{width}}_{W_2})$ ⁶

第_{i₁}行 第_{i₂}列

N_2 : 8列数

- k-dimensional array⁹

◆ $\text{addr}(A[i_1][i_2] \dots [i_k]) = \text{base} + i_1 \times W_1 + i_2 \times W_2 + \dots + i_k \times W_k$ ¹⁰



Translation of Array References¹

• $\text{Type}(a) = \text{array}(10, \text{int})$ ²

◆ $c = a[i];$ ³

$\text{addr}(a[i]) = \text{base} + i * 4$ ⁴

$t_1 = i * 4$ ⁵

$t_2 = a[t_1]$ ⁶

$c = t_2$ ⁸

3行5列⁷

• $\text{Type}(a) = \text{array}(3, \text{array}(5, \text{int}))$ ⁹

◆ $c = a[i_1][i_2];$ ¹⁰

$\text{addr}(A[i_1, i_2]) = \text{base} + (i_1 \times N_2 * \text{width} + i_2 \times \text{width})$ ¹¹

$\text{addr}(a[i_1][i_2]) = \text{base} + i_1 * 20 + i_2 * 4$ ¹²

$t_1 = i_1 * 20$ ¹³

$t_2 = i_2 * 4$ ¹⁴

$t_3 = t_1 + t_2$ ¹⁶

$t_4 = a[t_3]$ ¹⁸

$c = t_4$ ²⁰

3个5行8列¹⁵

• $\text{Type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{int})))$ ¹⁷

◆ $c = a[i_1][i_2][i_3]$ ¹⁹

$\text{addr}(a[i_1][i_2][i_3]) = \text{base} + i_1 * w_1 + i_2 * w_2 + i_3 * w_3$ ²¹
 $= \text{base} + i_1 * 160 + i_2 * 32 + i_3 * 4$

160: a complete 2D array



Translation of Array References (cont.)



• $A[i1][i2][i3]$, $\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{int})))$ ²

◆ $L.\text{array}$: a pointer to the symbol-table entry for the array name³

□ $L.\text{array}.\text{base}$ gives the array's base address

◆ $L.\text{type}$: the type of the subarray generated by L ⁴

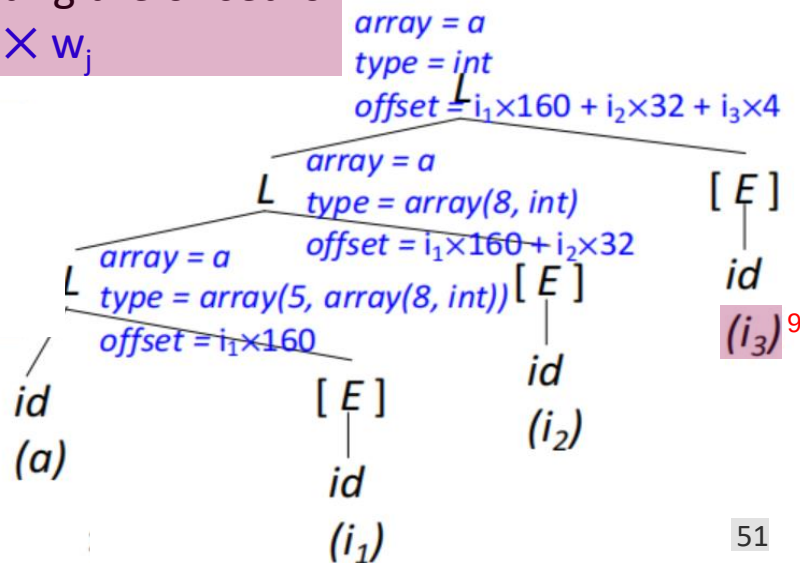
◆ $L.\text{addr}$: a temporary that is used while computing the offset for the array referenced by summing the terms $i_j \times w_j$ ⁵

① $S \rightarrow \text{id} = E; \mid L = E;$

② $E \rightarrow E_1 + E_2 \mid - E_1 \mid (E_1) \mid \text{id} \mid L$ ⁶

③ $L \rightarrow \text{id} [E] \mid L_1 [E]$ ⁷

$\text{base} + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k$ ⁸

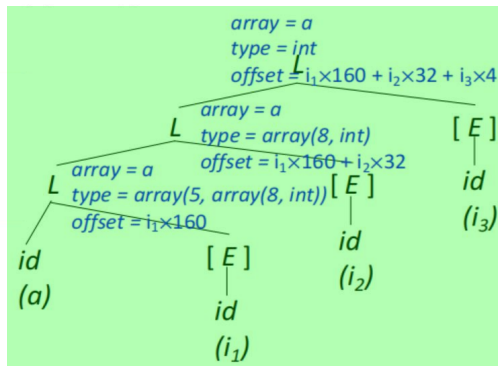


Translation of Array References (cont.)



• $A[i_1][i_2][i_3]$, $\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{int})))$

- ① $S \rightarrow \text{id} = E; \mid L = E; \{ \text{gen}(L.\text{array}.\text{base}['L.\text{addr}'] \text{ '=' } E.\text{addr}); \}$
- ② $E \rightarrow E_1 + E_2 \mid - E_1 \mid (E_1) \mid \text{id} \mid L \{ E.\text{addr} = \text{newtemp}(); \text{gen}(E.\text{addr} \text{ '=' } L.\text{array}.\text{base}['L.\text{addr}']); \}$
- ③ $L \rightarrow L_1 [E] \{ L.\text{array} = L_1.\text{array}; L.\text{type} = L_1.\text{type}.\text{elem}; t = \text{newtemp}(); \text{gen}(t \text{ '=' } E.\text{addr} \text{ '*' } L.\text{type}.\text{width}); L.\text{addr} = \text{newtemp}(); \text{gen}(L.\text{addr} \text{ '=' } L_1.\text{addr} \text{ '+' } t); \} \mid$
 $\text{id} [E] \{ L.\text{array} = \text{lookup}(\text{id}.\text{lexeme}); \text{if } !L.\text{array} \text{ then error}; L.\text{type} = L.\text{array}.\text{type}.\text{elem}; L.\text{offset} = \text{newtemp}(); \text{gen}(L.\text{addr} \text{ '=' } E.\text{addr} \text{ '*' } L.\text{type}.\text{width}); \}$



$t_1 = i_1 * 160$ $t_4 = i_3 * 4$
 $t_2 = i_2 * 32$ $t_5 = t_3 + t_4$
 $t_3 = t_1 + t_2$ $c = a[t_5]$



Code Generation¹[代码生成]

- We will use the **syntax-directed formalisms** to specify translation²

- ◆ Variable definitions[变量定义] -> Recall semantic analysis & symbol table³

- ◆ Assignment[赋值]⁴

- ◆ Array references[数组引用]⁵

- ◆ Boolean expressions[布尔表达式]⁶

- ◆ Control-flow statements[控制流语句]⁷

- To generate three-address codes (TACs)⁸

- ◆ Lay out variables in memory⁹

- ◆ Generate TAC for any subexpressions or substatements¹⁰

- ◆ Using the result, generate TAC for the overall expression¹¹



CodeGen: Boolean Expressions¹

- Boolean expression: `a op b`²

◆ where op can be `<`, `<=`, `!=`, `>`, `>=`, `&&`, `||`, `==`, ...³

- Short-circuit evaluation[短路计算]: to skip evaluation of the rest of a boolean expression once a boolean value is known⁴

◆ Given following C code: `if (flag || foo()) { bar(); };`⁵

□ If `flag` is true, `foo()` never executes⁶

□ Equivalent to: `if (flag) { bar(); } else if (foo()) { bar(); };`⁷

◆ Given following C code: `if (flag && foo()) { bar(); };`⁸

□ If `flag` is false, `foo()` never executes⁹

□ Equivalent to: `if (!flag) { } else if (foo()) { bar(); };`¹⁰

- For control flow, boolean operators is translated to jump statements¹¹

Boolean Expressions¹

- Computed just like any other arithmetic expression²

$E \rightarrow (a < b) \text{ or } (c < d \text{ and } e < f)$ ³

$t_1 = a < b$ ⁴

$t_2 = c < d$ ⁵

$t_3 = e < f$ ⁶

$t_4 = t_2 \ \&\& \ t_3$ ⁷

$t_5 = t_1 \ || \ t_4$ ⁸

- Then, used in control-flow statements⁹

◆ **S.next**: label for code generated after S¹⁰

$S \rightarrow \text{if } E \ S_1$ ¹¹

$\text{if } (!t_5) \text{ goto } S.\text{next}$ ¹²

$S_1.\text{code}$ ¹³

$S.\text{next}: \dots$ ¹⁴



Boolean Expressions¹

- Implemented via a series of **jumps**[利用跳转]²

- ◆ converted to **two gotos** (true and false)³

- ◆ Remaining evaluation skipped when result known in middle⁴

- Example⁵

- ◆ **E.true**: label for code to execute when E is 'true'⁶

- ◆ **E.false**: label for code to execute when E is 'false'⁷

- ◆ E.g. if above is condition for a do-while loop⁸

- **E.true** would be label at **beginning of loop body**⁹

- **E.false** would be label for **code after the loop**¹⁰

$E \rightarrow (a < b) \text{ or } (c < d \text{ and } e < f)$ ¹¹

if $(a < b)$ goto **E.true**

goto **L₁**¹²

L₁: if $(c < d)$ goto **L₂**

goto **E.false**¹³

L₂: if $(e < f)$ goto **E.true**

goto **E.false**¹⁴

E为真: 只要 $a < b$ 真¹⁵

$a < b$ 假: 继续评估¹⁶

$a < b$ 假、 $c < d$ 真: 继续评估¹⁷

E为假: $a < b$ 假, $c < d$ 假¹⁸

E为真: $a < b$ 假, $c < d$ 真, $e < f$ 真¹⁹

E为假: $a < b$ 假, $c < d$ 真, $e < f$ 假²⁰

Boolean Expressions¹

- Boolean expressions are composed of²

- ◆ Boolean operators (`==`, `&&`, `||`) applied to elements that are boolean variables or relational expressions (`E1 relop E2`)³

- Computed just like any other arithmetic expression⁶

$E \rightarrow (a < b) \text{ or } (c < d \text{ and } e < f)$ ⁹

$t_1 = a < b$ ⁴

$t_2 = c < d$ ⁵

$t_3 = e < f$ ⁷

$t_4 = t_2 \&\& t_3$ ⁸

$t_5 = t_1 || t_4$ ¹⁰

- Then, used in control-flow statements¹¹

- ◆ `S.next`: label for code generated after `S`¹²

`if (! $t_5$ ) goto  $S.next$` ¹³

`$S_1$ .code`¹⁴

`$S.next$ : ...`¹⁶

$S \rightarrow \text{if } E \text{ } S_1$ ¹⁵



SDT Translation of Booleans[布尔表达式]¹



- $B \rightarrow B1 \ || \ B2$ ²

- ◆ $B1.true$ is same as $B.true$, $B2$ must be evaluated if $B1$ is false[B1假才评估B2]³

- ◆ The true and false exits of $B2$ are the same as B [B2与B同真假]⁴

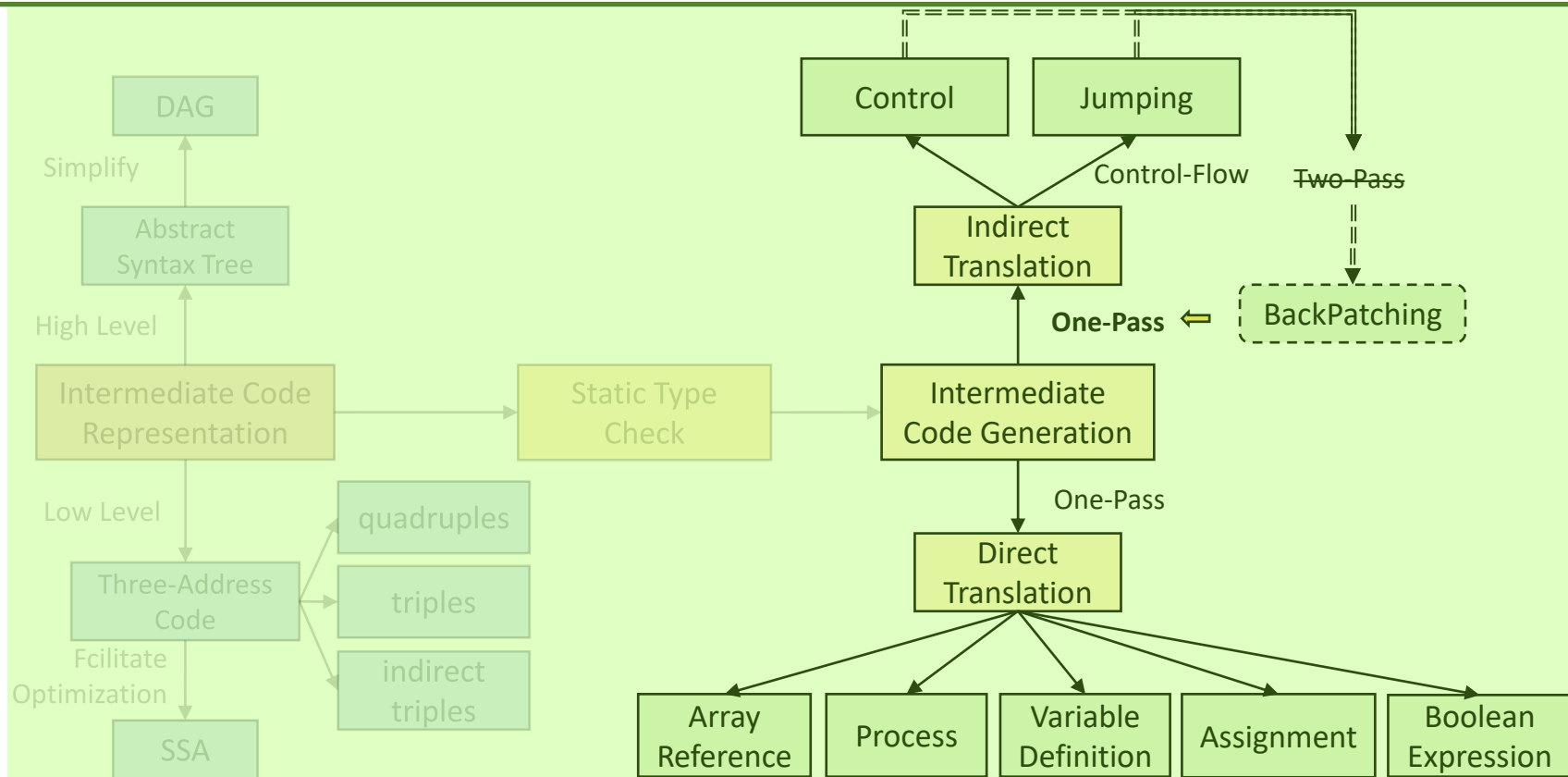
- $B \rightarrow E1 \text{ relop } E2$ ⁵

- ◆ Translated directly into a comparison TAC inst with jumps⁶

- ① $B \rightarrow \{ B1.true = B.true; B1.false = \text{newlabel}(); \} B1 \ || \ \{ \text{label}(B1.false); B2.true = B.true; B2.false = B.false; \} B2$ ⁷
- ② $B \rightarrow \{ B1.true = \text{newlabel}(); B1.false = B.false; \} B1 \ \&\& \ \{ \text{label}(B1.true); B2.true = B.true; B2.false = B.false; \} B2$
- ③ $B \rightarrow E1 \text{ relop } E2 \ \{ \text{gen}('if' \ E1.addr \ \text{relop} \ E2.addr \ 'goto' \ B.true); \text{gen}('goto' \ B.false); \}$
- ④ $B \rightarrow ! \ \{ B1.true = B.false; B1.false = B.true; \} B1$
- ⑤ $B \rightarrow \text{true} \ \{ \text{gen}('goto' \ B.true); \}$
- ⑥ $B \rightarrow \text{false} \ \{ \text{gen}('goto' \ B.false); \}$



Intermediate Code[中间代码生成]¹



2

Code Generation¹[代码生成]

- We will use the **syntax-directed formalisms** to specify translation²

- ◆ Variable definitions[变量定义] -> Recall semantic analysis & symbol table³

- ◆ Assignment[赋值]⁴

- ◆ Array references[数组引用]⁵

- ◆ Boolean expressions[布尔表达式]⁶

- ◆ Control-flow statements[控制流语句]⁷

- To generate three-address codes (TACs)⁸

- ◆ Lay out variables in memory⁹

- ◆ Generate TAC for any subexpressions or substatements¹⁰

- ◆ Using the result, generate TAC for the overall expression¹¹



CodeGen: Control Statement[控制语句]¹

- Inherited attributes[继承属性]²

- ◆ **B.true**: the label to which control flows if B is true (依赖于S1)³

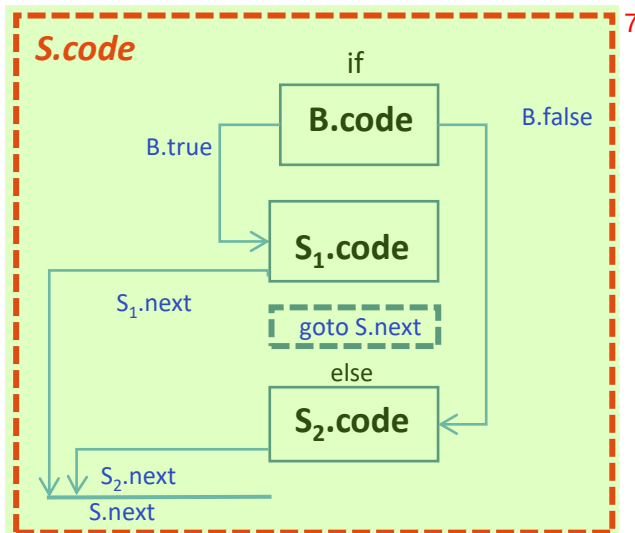
- ◆ **B.false**: the label to which control flows if B is false (依赖于S2)⁴

- ◆ **S.next**: a label for the instruction immediately after the code of S⁵

① $S \rightarrow \text{if} (B) S_1$

② $S \rightarrow \text{if} (B) S_1 \text{ else } S_2$

③ $S \rightarrow \text{while} (B) S_1$



Translation of Controls¹

• Helper functions[辅助函数]²

◆ `newlabel()`: creates a new label³

◆ `label(L)`: attaches label L to the next three address inst to be generated⁴

① `S -> if ( B ) S1`⁵

② `S -> if ( B ) S1 else S2`

③ `S -> while ( B ) S1`

`S -> if { B.true = newlabel();
B.false = newlabel(); }
(B) { label(B.true); S1.next = S.next; }
S1 { gen('goto' S.next); }
else { label(B.false); S2.next = S.next; } S2`⁶

`If false B goto B.false`⁷

`B.true:`

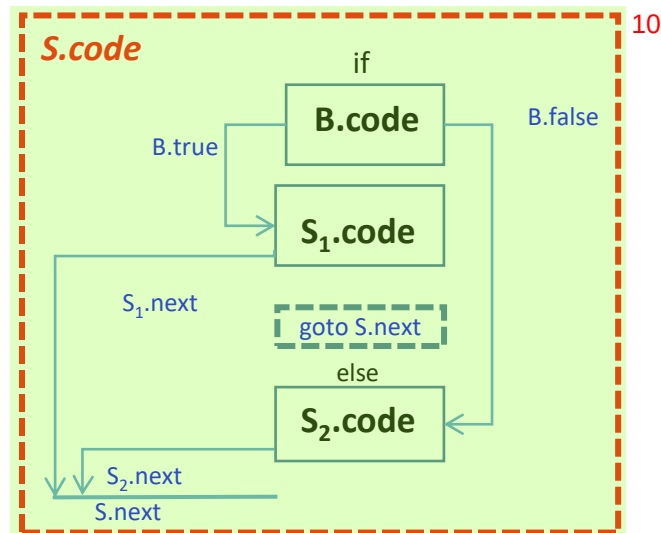
`S1.code`

`goto S.next`

`B.false:`⁸

`S2.code`

`S.next:`⁹



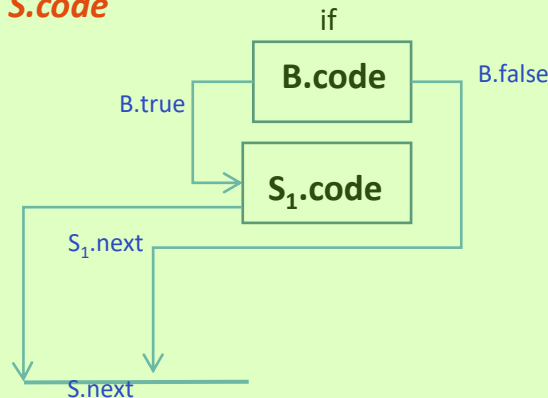
Translation of Controls (cont.)¹

- ① $S \rightarrow \text{if } (B) S_1$
- ② $S \rightarrow \text{if } (B) S_1 \text{ else } S_2$
- ③ $S \rightarrow \text{while } (B) S_1$

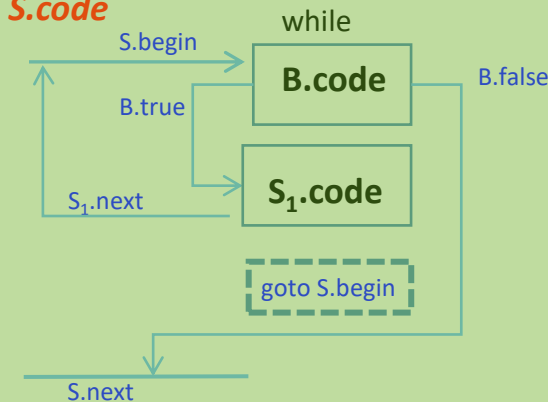
$S \rightarrow \text{if } \{ B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next}; \}$
 $(B) \{ \text{label}(B.\text{true}); S_1.\text{next} = S.\text{next}; \}$
 S_1

$S \rightarrow \text{while } \{ S.\text{begin} = \text{newlabel}();$
 $\text{label}(S.\text{begin});$
 $B.\text{true} = \text{newlabel}();$
 $B.\text{false} = S.\text{next}; \}$
 $(B) \{ \text{label}(B.\text{true}); S_1.\text{next} = S.\text{begin}; \}$
 $S_1 \{ \text{gen('goto' } S.\text{begin}); \}$

S.code



S.code



Jumping Labels[跳转标签]¹

- Key of generating code for Boolean and flow-control: **matching a jump** inst with the **target of jump**[跳转指令匹配到跳转目标]²

- ◆ Forward jump: a jump to an instruction in below³

- ◆ Label for jump target has not yet been generated⁴

```
B -> { B1.true = newlabel(); B1.false = B.false; } B1 && { label(B1.true); B2.true = B.true; B2.false = B.false; } B25  
S -> if { B.true = newlabel(); B.false = S.next; } ( B ) { label(B.true); S1.next = S.next; } S1
```



Handle Jumping Labels¹

- Idea: generate code using dummy labels first, then patch them with addresses later after labels are generated²

- **Two-pass** approach: requires two scans of code³

- ◆ Pass 1:⁴

- Generate code creating **dummy labels** for forward jumps. (Insert label into a **hashtable**)⁵
- When label emitted, **record address** in hashtable⁶

- ◆ Pass 2:⁷

- **Replace dummy labels** with target addresses (Use previously built hashtable for mapping)⁸

- **One-pass** approach⁹

- ◆ Generate **holes** when forward jumping to an un-generated label¹⁰
- ◆ **Maintain a list of holes** for that label¹¹
- ◆ **Fill in holes with addresses** when label generated later on¹²

One-Pass Code Generation¹[单遍生成]

- One Pass Generation takes less time along with LR parser²
- However, given the example below, we need to know the address³ of **E2.label** to insert jumps in **E1**
 - ◆ E.g. **E1.false = E2.label** in $E \rightarrow E1 \mid\mid E2$ ⁴
- Solution: **Backpatching**[回填]⁵
 - ◆ Leave **holes** in IR in place of forward jump addresses⁶
 - ◆ Record indices of jump instructions in **a hole list**⁷
 - ◆ When target address of label for jump is eventually known, **backpatch**⁸ **holes** using the hole list for that particular label



Backpatching[回填]¹

- Synthesized attributes[综合属性]. $S \rightarrow \text{if } (B) S1$ ²

- ◆ **B.truelist**: a list of jump or conditional jump insts into which we must insert the label to which control goes if B is true[B为真时控制流应该转向的指令的标号]³
- ◆ **B.falselist**: a list of insts that eventually get the label to which control goes when B is false[B为假时控制流应该转向的指令的标号]⁴
- ◆ **S.nextlist**: a list of jumps to the inst immediately following the code for S[紧跟在S代码之后的指令的标号]⁵

- Helper functions to implement backpatching⁶

- ◆ **makelist(i)**: creates a new list out of statement index **i**⁷
- ◆ **merge(p1, p2)**: returns merged list of **p1** and **p2**⁸
- ◆ **backpatch(p, i)**: fill holes in list **p** with statement index **i**⁹



Backpatching of Control-Flow¹

• Slightly modify the grammar²

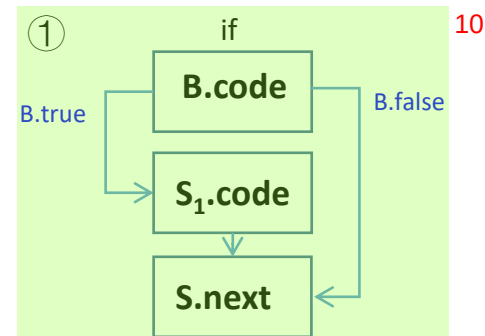
① $S \rightarrow \text{if } (B) \ M \ S_1 \ \{ \text{backpatch}(B.\text{truelist}, M.\text{inst})$
 $\quad S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \}$ ³

② $S \rightarrow \text{if } (B) \ M_1 \ S_1 \ N \ \text{else} \ M_2 \ S_2 \ \{ \text{backpatch}(B.\text{truelist}, M_1.\text{inst});$
 $\quad \text{backpatch}(B.\text{falselist}, M_2.\text{inst});$
 $\quad \text{temp} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist});$
 $\quad S.\text{nextlist} = \text{merge}(\text{temp}, S_2.\text{nextlist}); \}$ ⁴

③ $S \rightarrow \text{while } M_1 \ (B) \ M_2 \ S_1 \ \{ \text{backpatch}(B.\text{truelist}, M_2.\text{inst});$
 $\quad \text{backpatch}(S_1.\text{nextlist}, M_1.\text{inst});$
 $\quad S.\text{nextlist} = B.\text{falselist};$
 $\quad \text{gen}(\text{'goto' } M_1.\text{inst}); \}$ ⁵

④ $M \rightarrow \epsilon \ \{ M.\text{inst} = \text{nextinst}; \}$ ⁶
⑤ $N \rightarrow \epsilon \ \{ N.\text{nextlist} = \text{makelist}(\text{nextinst}); \text{gen}(\text{'goto' } _); \}$

- **makelist(i)**: creates a new list out of statement index i⁷
- **merge(p1, p2)**: returns merged list of p1 and p2⁸
- **backpatch(p, i)**: fill holes in list p with statement index i⁹



Backpatching of Control-Flow¹

• Slightly modify the grammar²

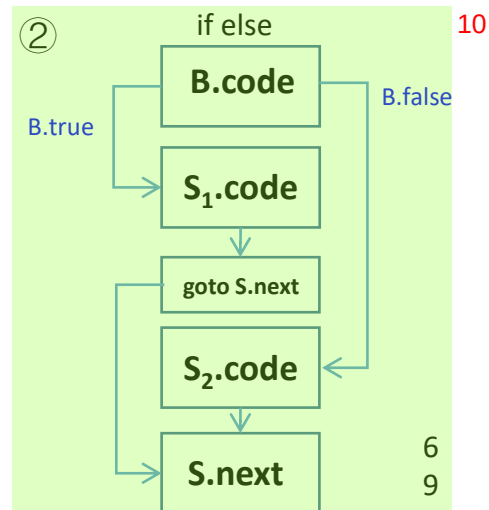
① $S \rightarrow \text{if } (B) \ M \ S_1 \ \{ \text{backpatch}(B.\text{truelist}, M.\text{inst})$
 $\quad S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \}$ ³

② $S \rightarrow \text{if } (B) \ M_1 \ S_1 \ N \ \text{else} \ M_2 \ S_2 \ \{ \text{backpatch}(B.\text{truelist}, M_1.\text{inst});$
 $\quad \text{backpatch}(B.\text{falselist}, M_2.\text{inst});$
 $\quad \text{temp} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist});$
 $\quad S.\text{nextlist} = \text{merge}(\text{temp}, S_2.\text{nextlist}); \}$ ⁴

③ $S \rightarrow \text{while } M_1 \ (B) \ M_2 \ S_1 \ \{ \text{backpatch}(B.\text{truelist}, M_2.\text{inst});$
 $\quad \text{backpatch}(S_1.\text{nextlist}, M_1.\text{inst});$
 $\quad S.\text{nextlist} = B.\text{falselist};$
 $\quad \text{gen}(\text{'goto' } M_1.\text{inst}); \}$ ⁵

④ $M \rightarrow \epsilon \ \{ M.\text{inst} = \text{nextinst}; \}$ ⁶
⑤ $N \rightarrow \epsilon \ \{ N.\text{nextlist} = \text{makelist}(\text{nextinst}); \text{gen}(\text{'goto' } _); \}$

- $\text{makelist}(i)$: creates a new list out of statement index i ⁷
- $\text{merge}(p_1, p_2)$: returns merged list of p_1 and p_2 ⁸
- $\text{backpatch}(p, i)$: fill holes in list p with statement index i ⁹



Backpatching of Control-Flow¹

• Slightly modify the grammar²

① $S \rightarrow \text{if } (B) \ M \ S_1 \ \{ \text{backpatch}(B.\text{truelist}, M.\text{inst})$
 $\quad S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \}$ ³

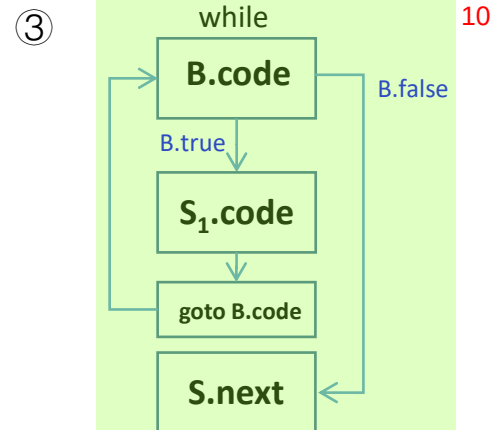
② $S \rightarrow \text{if } (B) \ M_1 \ S_1 \ N \ \text{else} \ M_2 \ S_2 \ \{ \text{backpatch}(B.\text{truelist}, M_1.\text{inst});$
 $\quad \text{backpatch}(B.\text{falselist}, M_2.\text{inst});$
 $\quad \text{temp} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist});$
 $\quad S.\text{nextlist} = \text{merge}(\text{temp}, S_2.\text{nextlist}); \}$ ⁴

③ $S \rightarrow \text{while } M_1 \ (B) \ M_2 \ S_1 \ \{ \text{backpatch}(B.\text{truelist}, M_2.\text{inst});$
 $\quad \text{backpatch}(S_1.\text{nextlist}, M_1.\text{inst});$
 $\quad S.\text{nextlist} = B.\text{falselist};$
 $\quad \text{gen}(\text{'goto' } M_1.\text{inst}); \}$ ⁵

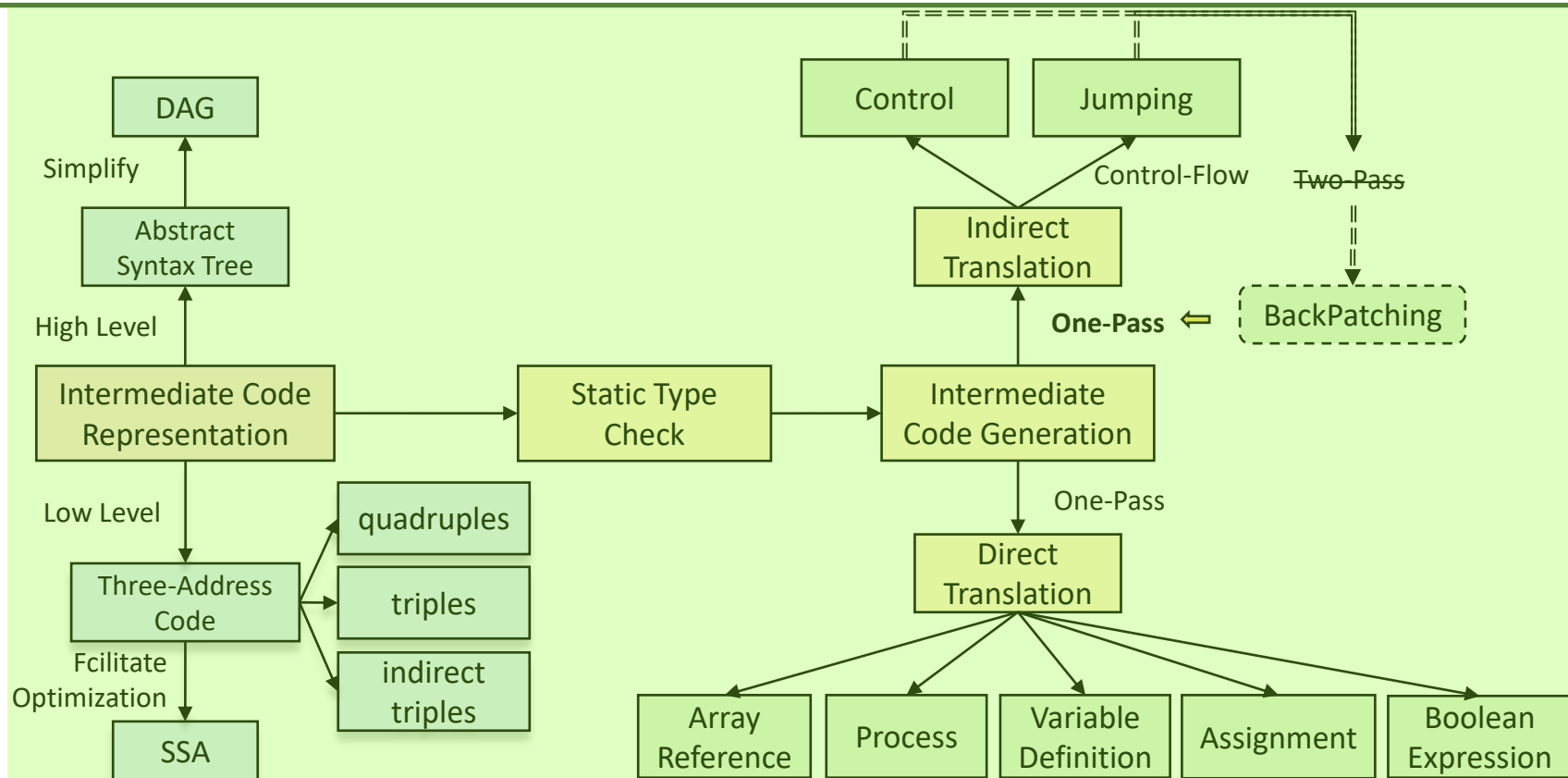
④ $M \rightarrow \epsilon \ \{ M.\text{inst} = \text{nextinst}; \}$ ⁶

⑤ $N \rightarrow \epsilon \ \{ N.\text{nextlist} = \text{makelist}(\text{nextinst}); \text{gen}(\text{'goto' } _); \}$

- **makelist(i)**: creates a new list out of statement index i⁷
- **merge(p1, p2)**: returns merged list of p1 and p2⁸
- **backpatch(p, i)**: fill holes in list p with statement index i⁹



Intermediate Code[中间代码生成]¹



2

- Three-Address Code: $X = Y \text{ op } Z$ ²

- ◆ Three representations³

- quadruples [四元式]⁴

- triples [三元式]⁵

- indirect triples [间接三元式]⁶

- Single Static Assignment⁷

- Code generation: TAC instructions using syntax directed translation⁸

- ◆ Variable definitions[变量定义]⁹

- ◆ Expressions and statements¹⁰

- Assignment[赋值]¹¹

- Array references[数组引用]¹²

- Boolean expressions[布尔表达式]¹³

- Control-flow[控制流]¹⁵



Further Reading¹

• Dragon Book, 2nd Edition²

◆ Comprehensive Reading:³

- Section 6.2 on introduction to intermediate representations.⁴
- Section 6.5 on type checking.⁵
- Section 6.3, 6.4, 6.6 and 6.7 on translations of various program constructs.⁶

◆ Skip Reading:⁷

- Section 6.1 on AST and DAG.⁸
- Section 6.8 and 6.9 on translations of switches and procedures.⁹



10